Fireflies: Intrusion Tolerant and Scalable Membership Protocol

Release 0.25.1

Håvard Johansen

September 1, 2005

haavardj@cs.uit.no

Abstract

Contents

1 Introduction **1**

2 Requirements **2**

3 Installation **2**

4 Usage **2**
4.1 Setting up a group . . . 2
Generating CA keys . . . 2
Generating PublicCertificates . . . 3
4.2 Using Groups . . . 3
4.3 Subclassing BaseGroup . . . 3

5 Modules **3**
5.1 `fireflies.member.` — Membership classes. . . . 3
5.2 `fireflies.web` — Web Status . . . 4

1 Introduction

Fireflies is an intrusion tolerant group membership protocol that provides satisfactory scalability for most applications. With a high probability, our protocol provides the following guarantees:

- A correct member will eventually always appear correct to all correct members.
- A failed member will eventually always appear failed to all correct members.

2 Requirements

Fireflies is being developed on an Intel based Debian-Linux system. Compatibility with other system has not yet received priority. Even so, you should be able to use it on any Python enabled system.

The software required by Fireflies is listed below. The given version numbers are for those Fireflies is known to work. Other versions may or may not work.

Required software:

Python 2.3.5 (http://www.python.org/2.3.5/)

Twisted Core 2.0.1 (http://twistedmatrix.com/projects/core)

Python Cryptography Toolkit 2.0.1 (http://www.amk.ca/python/code/crypto)

Optional software for the `fireflies.web` module:

Nevow 0.4.1 (http://nevow.com/)

Twisted Web 0.5.0 (http://twistedmatrix.com/projects/web/)

Gnuplot 4.0 (http://www.gnuplot.info/) with build-in support for new-type PNG output.

Gnuplot.py 1.7 (http://gnuplot-py.sourceforge.net/)

3 Installation

After installing all the required packages, install Fireflies by:

1. Download 'fireflies-0.25.1.tar.gz' and untar the package.
2. In the 'fireflies-0.25.1/' directory, run `python setup.py install`.

This will install Fireflies into the standard Python site-packages path. You can specify a **–prefix=PATH** options to the `setup.py` script to install it in a different path.

4 Usage

4.1 Setting up a group

A group is defined by a Certificate Authority's (CA's) public key. This key must be available to all members. Each member is represented in a group by a `PublicCertificate` that is signed by the CA's private key. The `PublicCertificate` binds the member's network address to his public key.

Generating CA keys

To generate the CA's private key, use the `generateKey` method in the `fireflies.cert` module. The public key can be extracted by using the private key's `public_key` method.

Running the included **ffcert** without any parameters will generate a private and public key pair, pickled to files.

Generating PublicCertificates

The `createCertificate` in the `fireflies.cert` module will create new private key `PublicCertificate` tuples. The name parameter to this function should be an ip-address:port string.

The included **ffcert** can produce pickled `PublicCertificate` objects. For example

```
#> ffcert -c ca.priv 192.168.2.1:12131
```

4.2 Using Groups

4.3 Subclassing BaseGroup

By subclassing the `fireflies.BaseGroup` class, your get access to group events like joins and leaves.

5 Modules

5.1 `fireflies.member.` — Membership classes.

class `Accusation`(*accused, epoch, ringnr, accuser*[, *signature*])
A Fireflies accusation. The *accused* and *accuser* parameters must be some members's serial number.

`accused`
The serial number of the member who is accused. **Note:** Read only.

`accuser`
The serial number of the member who is accusing. **Note:** Read only.

`nr`
The ring that the accusation is made on. **Note:** Read only.

`lifeline`
The accused's lifeline. **Note:** Read only.

`signature`
Objects signature.

`sing`(*key*)
Sign the object using the `Crypto.RSA` private *key*.

`verify`(*key*)
Returns `True` if the object is signed by *key*'s matching private key. *key* must be either a `Crypto.RSA` object, or a `PublicCertificate` object.

`uidstr`()
Returns an unique string representation of the object.

`shash`()
Returns a secure-hash integer of the object.

class `Note`(*serial*[, *lifeline*[, *mask*[, *signature*[, *data*]]]])
A Fireflies note.

`serial`
The member's serial number. **Note:** Read only.

`lifeline`
The members lifeline. **Note:** Read only.

mask
The mask is an array of `True` or `False` values. A `True` value means that the corresponding monitoring ring is enabled. **Note:** Read only.

data
Data **Note:** Read only.

signature
Objects signature.

setTimestamp()
Timestamps the accusation with the time as returned by `fireflies.time()`.

getTimestamp()
Returns the latest timestamp, or 0 if `setTimestamp` has not yet been called.

sing(*key*)
Sign the object using the `Crypto.RSA` private *key*.

verify(*key*)
Returns `True` if the object is signed by *key*'s matching private key. *key* must be either a `Crypto.RSA` object, or a `PublicCertificate` object.

uidstr()
Returns an unique string representation of the object.

shash()
Returns a secure-hash integer of the object.

5.2 `fireflies.web` — Web Status

This module provides a web based node status display.

class NodeViewer(*group*[, *port*[, *interface*]])
Display the status of a `firefles.group.BaseGroup` derivate as given by the *group* parameter. The optional *port* and *interface* can be used to bind the server to a specific port and host. If these are not supplied, the server will bind to the interface as and port + 1000 as given in `group.peer.cert.address`.

start()
Starts listening for incoming http requests.